Classical and Transformer Models for Multi-label Code Comment Classification: An Exploratory Study on NLBSE'23

Abstract

Class comment sentences can convey several information types, making their automatic categorization a multi-label learning problem. This study compares five classical machine-learning methods and seventeen RoBERTa/CodeBERT configurations on 6,738 NLBSE'23 sentences from Python, Java, and Pharo. Existing experimental records are synthesized under shared, exact-text-grouped partitions: five-fold screening followed by fixed approximately 80/10/10 training, validation, and test splits. CodeBERT with R-Drop has the highest mean screening micro-F1 on Python and Java, at 64.79% and 62.25%; RoBERTa with focal loss leads Pharo at 71.52%. Their differences from the RoBERTa binary-cross-entropy reference are 3.10, 2.57, and 1.07 percentage points. Classical models remain competitive on Java, whereas Transformer gains are larger on Pharo. Per-label analysis shows that aggregate improvements can coexist with weaker minority-label performance. Screening used the full population, and neural checkpoints were selected on the same fold used for scoring; consequently, the reported final-test results are exploratory rather than independent confirmation. The contribution is a traceable comparison of completed experiments and their evaluation limits, not a new architecture or a claim of competition-level superiority. **Keywords:** code comments; multi-label classification; TF-IDF; RoBERTa; CodeBERT; cross-validation.

1 Introduction

Class comments provide information about purpose, usage, responsibilities, and implementation. A sentence can simultaneously express several of these roles. Assigning exactly one category would discard annotations and obscure the distinct information needs served by documentation. Earlier work investigated language-specific comment taxonomies and automated categorization across Python, Java, and Smalltalk [2]; the NLBSE'23 code-comment competition subsequently provided a common evaluation resource [1, 3].

Two modeling directions are relevant. Sparse lexical features with classical classifiers provide inexpensive baselines and expose the value of local word and character patterns. Pretrained Transformers can represent broader context, but their performance also depends on training objectives, label modeling, and checkpoint selection. A useful empirical comparison must therefore align the data and evaluation procedure before interpreting architectural differences.

We investigate three questions: **RQ1**, how do classical methods compare with a fixed RoBERTa reference? **RQ2**, which recorded neural configurations lead within each programming language? **RQ3**, do aggregate improvements also improve individual labels? The study synthesizes 66 five-fold evaluations and 30 final-stage evaluations of non-trivial methods. It separates classical ML from DL and then compares them under a common protocol. No additional training, ensemble fitting, or hyperparameter search is introduced for this paper.

2 Background and study scope

Binary relevance learns one classifier per label; classifier chains additionally condition a label on earlier labels [4]. RoBERTa [5] provides a pretrained text encoder, while CodeBERT [6] incorporates natural-language and programming-language pretraining. Focal loss reweights classification errors according to prediction difficulty [7]; R-Drop encourages agreement between two stochastic dropout passes [8]. These mechanisms motivate the evaluated variants but do not imply that a variant must improve this dataset.

The 6,738 examples belong to the *code-comment* task, rather than the separate NLBSE'23 issue-report task. The official code-comment baselines are per-category Random Forest classifiers with lexical and NLP-derived features [3]. Our RF uses a different feature representation and newly constructed splits; it is not an exact reproduction of that baseline. RoBERTa with binary cross-entropy, denoted E1, is the fixed *Transformer reference within this study*. All numerical comparisons below concern this shared protocol, not the official competition leaderboard.

3 Data and experimental protocol

3.1 Sentence-level multi-label targets

The long-format data are consolidated by sentence identifier, preserving all binary annotations. A sentence x_i has a target vector $\mathbf{y}_i \in \{0,1\}^L$, where L is five for Python and seven each for Java and Pharo. Label names are language-specific; models are trained separately, rather than pooling nineteen categories into a shared prediction task.

Only whitespace is normalized before grouping. Identical normalized texts remain in the same fold or final partition, even if their sentence identifiers differ. There are 2,299, 1,668, and 1,620 distinct text groups for Python, Java, and Pharo. Thus, repeated texts are sufficiently common to make row-wise random splitting inappropriate. Targets are retained per identifier; differing annotations for identical text are not automatically treated as annotation errors.

Table 1 describes the population. Label imbalance is substantial: Java *Ownership* has 115 positives compared with 912 for *usage*; Pharo *Classreferences* has 77 compared with 748 for *Example*. Because mean label cardinality is near one while the label space contains five or seven categories, negative targets dominate. Both label-sensitive and aggregate metrics are needed.

3.2 Two-stage evaluation

Stage 1: screening. All sentences participate in five grouped folds. A group-balancing procedure approximately balances sample and positive-label counts; it is not guar-

Table 1: Dataset population and final-stage partitions. Cardinality is the mean number of positive labels per sentence.

Language	Sentences	Labels	Train	Validation	Test	Cardinality
Python	2555	5	2044	255	256	1.121
Java	2418	7	1935	241	242	1.177
Pharo	1765	7	1413	176	176	1.158

anteed to achieve exact stratification. Each outer training portion fits a fresh feature extractor/classifier or initializes a fresh neural model from pretrained weights. Classical TF-IDF statistics, class weights, and neural label-graph statistics are estimated only from that fold's training portion. Methods share fold assignments and label order within each language. Mean and sample standard deviation (SD) are calculated from the five fold scores; SD is not a confidence interval.

Stage 2: recorded final evaluations. The same population is independently arranged into approximately 80% train, 10% validation, and 10% test, respecting exact-text groups. Classical methods fit once on train and use validation only diagnostically. Neural models use validation micro-F1 to select checkpoints before test evaluation. Five classical methods and E1/E3/E9/E15/E17 have final evaluations in every language. This subset does not exactly cover each language's four highest-ranked neural configurations: E13/E11 are missing for Python, E12 for Java, and E8/E12 for Pharo. Missing evaluations are not imputed.

Interpretation constraint. Stage-1 screening already uses every sentence, including those later assigned to Stage-2 test. Therefore, Stage-2 test results are not independent of method selection. Moreover, each DL fold uses its validation data for both checkpoint selection and fold scoring. These are completed screening experiments, not nested-CV estimates or a prospective held-out benchmark. We select representative methods by mean Stage-1 micro-F1 and keep that choice fixed when examining Stage 2.

3.3 Metrics

For label j, let TP_j, FP_j, and FN_j denote prediction counts. We use

$$F_{1,\text{micro}} = \frac{2\sum_j TP_j}{2\sum_j TP_j + \sum_j FP_j + \sum_j FN_j}, \quad (1)$$

$$F_{1,\text{macro}} = \frac{1}{L}\sum_{j=1}^{L} \frac{2TP_j}{2TP_j + FP_j + FN_j}, \quad (2)$$

with an undefined label F1 set to zero. Hamming loss is

$$HL = \frac{1}{nL}\sum_{i=1}^{n}\sum_{j=1}^{L} \mathbb{1}[y_{ij} \neq \hat{y}_{ij}]. \quad (3)$$

Micro-F1 is the primary ranking metric; macro-F1 exposes differences across labels, and lower HL indicates fewer target-level errors. Micro average precision (AP) summarizes the ranking of all sentence–label scores without selecting a classification threshold. SVM decision scores are suitable for AP as ranking scores, but they are not calibrated probabilities. Comparisons across languages remain descriptive because label spaces and sample distributions differ.

4 Evaluated methods

4.1 Classical machine learning

All five methods use the concatenation of word unigram/bigram TF-IDF and character-within-word 3–5-gram TF-IDF. Each branch retains at most 20,000 features with minimum document frequency two. TF-IDF uses default lowercasing, while exact-text grouping preserves case. Vocabulary and IDF are learned only on the relevant training portion.

LR is one-vs-rest logistic regression with $C = 5$, balanced class weights, and an L-BFGS iteration cap of 2,000. **Chain** uses the same logistic base estimator in a classifier chain with one seeded random label order; no chain ensemble is evaluated. **SVM** is one-vs-rest Linear SVM with $C = 1$ and balanced weights. **SGD** uses one-vs-rest logistic-loss classifiers, balanced weights, an iteration cap of 2,000, and tolerance 10^{-3}. **RF** uses one-vs-rest Random Forests with 300 trees and balanced weights. These are recorded fixed settings, not the result of a new grid search. The majority Dummy control is excluded from the paper's comparative tables; the original measurements remain intact.

4.2 Deep learning

The neural reference maps a 768-dimensional first-token representation to L logits and applies independent sigmoids. The base objective is mean binary cross-entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{nL}\sum_{i,j}\left[y_{ij}\log p_{ij} + (1-y_{ij})\log(1-p_{ij})\right]. \quad (4)$$

RoBERTa configurations comprise E1 (BCE), E2 (positive-class-weighted BCE), E3 (focal), E4 (deferred threshold tuning), E5 (label attention), E6 (attention with contrastive regularization), E7 (asymmetric loss), E8 (R-Drop), E9 (label graph), E10 (adversarial embedding perturbations), and E12 (attention with a graph). CodeBERT configurations are E11 (BCE), E13 (separate head learning rate), E14 (layer-wise learning-rate decay), E15 (R-Drop), E16 (decay plus R-Drop), and E17 (focal).

E3/E17 use binary focal loss $-(1-p_t)^{\gamma}\log p_t$ with $\gamma = 2$ and no alpha weighting, where $p_t = p$ for a positive target and $1 - p$ otherwise. R-Drop uses two training passes and averages their supervised losses, adding a symmetric KL penalty between per-label Bernoulli distributions. In E15, its coefficient is 1.0. This multi-label consistency term does not apply a softmax across mutually nonexclusive labels.

The common training configuration uses batch size 16, at most ten epochs, maximum input length 128, AdamW with base learning rate 2×10^{-5}, weight decay 0.01, warmup ratio 0.1, linear scheduling, and gradient clipping at 1.0. Usual early-stopping patience is two; E9 uses patience three, at least five epochs, and AP as a validation tie-breaker. E13/E14/E16 use a 10^{-4} head learning rate; decay variants additionally reduce rates across encoder layers. The base

Table 2: Stage-1 micro-F1 (%, mean ± sample SD across five shared folds). Bold identifies the highest mean within the ML or DL family for each language. E1 is the Transformer reference. No significance claim follows from boldface.

Method	Python	Java	Pharo
LR	58.33 ± 1.26	**59.72 ± 5.66**	64.38 ± 2.23
Chain	56.23 ± 1.93	57.89 ± 6.75	62.09 ± 2.47
SVM	57.30 ± 0.95	58.11 ± 5.77	63.72 ± 2.10
SGD	58.11 ± 0.93	59.32 ± 5.56	**64.86 ± 1.88**
RF	**58.82 ± 1.26**	57.26 ± 4.05	64.58 ± 2.26
E1	61.69 ± 1.77	59.68 ± 6.88	70.44 ± 3.74
E2	61.61 ± 1.99	59.81 ± 5.93	69.12 ± 4.45
E3	62.19 ± 2.56	59.54 ± 5.90	**71.52 ± 2.33**
E4	61.69 ± 1.77	59.68 ± 6.88	70.44 ± 3.74
E5	60.80 ± 2.13	60.55 ± 4.88	69.92 ± 3.18
E6	61.58 ± 1.67	61.15 ± 6.88	70.36 ± 2.78
E7	60.81 ± 1.02	59.18 ± 5.64	67.36 ± 2.08
E8	62.71 ± 2.85	60.65 ± 5.21	71.04 ± 3.07
E9	62.11 ± 2.68	61.25 ± 5.55	71.24 ± 3.04
E10	62.60 ± 1.96	59.57 ± 4.62	69.50 ± 2.53
E11	63.97 ± 1.81	59.80 ± 3.00	68.05 ± 3.58
E12	61.32 ± 1.81	61.17 ± 5.19	70.68 ± 3.67
E13	64.55 ± 1.63	60.01 ± 2.84	68.93 ± 3.47
E14	63.07 ± 1.96	60.32 ± 2.05	67.67 ± 2.82
E15	**64.79 ± 2.37**	**62.25 ± 4.16**	69.56 ± 4.50
E16	63.76 ± 1.75	59.79 ± 4.04	67.48 ± 2.02
E17	64.52 ± 2.19	61.54 ± 2.59	69.93 ± 2.87

seed is 42. Classical methods run on CPU and neural models on GPU; total durations do not constitute a controlled speed comparison. Predictions use the recorded default rules, including probability threshold 0.5 for neural outputs. E4 uses 0.5 during screening and has no final-stage run, so no benefit of threshold tuning is established.

5 Results

5.1 Screening across all methods

RQ1: classical methods versus E1. RF leads the classical family on Python at $58.82 \pm 1.26\%$, LR leads Java at $59.72 \pm 5.66\%$, and SGD leads Pharo at $64.86 \pm 1.88\%$ (Table 2). E1 reaches 61.69%, 59.68%, and 70.44%, respectively. Thus, the best classical Java mean is essentially tied with E1: a difference of only 0.04 percentage points. The corresponding E1 advantages over the selected ML methods on Python and Pharo are approximately 2.87 and 5.58 points. These observations do not support a language-independent claim that DL always outperforms ML.

RQ2: leading neural configurations. E15, CodeBERT with R-Drop, leads Python ($64.79 \pm 2.37\%$) and Java ($62.25 \pm 4.16\%$). E3, RoBERTa with focal loss, leads Pharo ($71.52 \pm 2.33\%$). Relative to E1, the exact unrounded mean differences are 3.10, 2.57, and 1.07 percentage points. Differences are descriptive: shared folds enable aligned comparisons, but five dependent fold scores, one seed, and checkpoint selection do not establish statistical significance or a causal explanation.

CodeBERT pretraining alone does not explain all rankings: E11 is below E15 on Python and Java, while Pharo's leading method uses RoBERTa. This pattern motivates controlled follow-up studies of training objective and backbone interactions, rather than treating backbone choice as sufficient. Table 2 also shows that additional attention or graph structure does not consistently dominate the simpler reference.

5.2 Final-stage comparisons

The largest observed test micro-F1 values belong to Python E15 (65.41%), Java E17 (70.34%), and Pharo E1 (73.71%). These test maxima are reported descriptively and do not replace CV-based selection. Figure 1 instead follows the methods already selected in Stage 1.

Python E15 improves on E1 by 4.67 test micro-F1 points; macro-F1 rises from 55.47% to 59.65%, while HL falls from 16.56% to 14.37%. Micro-AP also rises from 65.71% to 75.85%, indicating that the observed improvement is not confined to hard predictions at a single threshold.

Java illustrates why more than one metric matters. Chain attains 69.64% micro-F1 and 74.70% macro-F1, compared with E1's 69.47% and 69.46%. Nevertheless, its HL is worse (10.04% versus 9.45%). E17 has the highest observed test micro-F1, whereas E15 remains the CV-selected neural method. Choosing E17 after observing test scores would change the selection rule.

On Pharo, E1 outperforms all four recorded neural alternatives at the default threshold. E3 was preferred by CV yet reaches only 69.19% test micro-F1 against E1's 73.71%. Their micro-AP values are closer, 79.56% and 79.90%. Ranking quality and thresholded classification therefore need not lead to identical interpretations; the present experiments do not isolate the cause of the rank reversal.

5.3 Per-label findings

RQ3: aggregate versus label-level improvement. Figure 2 shows heterogeneous effects. For Python, E15 improves *Usage* from 72.19% to 77.24%, *Parameters* from 64.60% to 71.43%, and *Summary* from 67.50% to 71.58%. In contrast, *DevelopmentNotes* decreases from 34.62% to 28.57%, despite the higher aggregate score. Its 31 positive

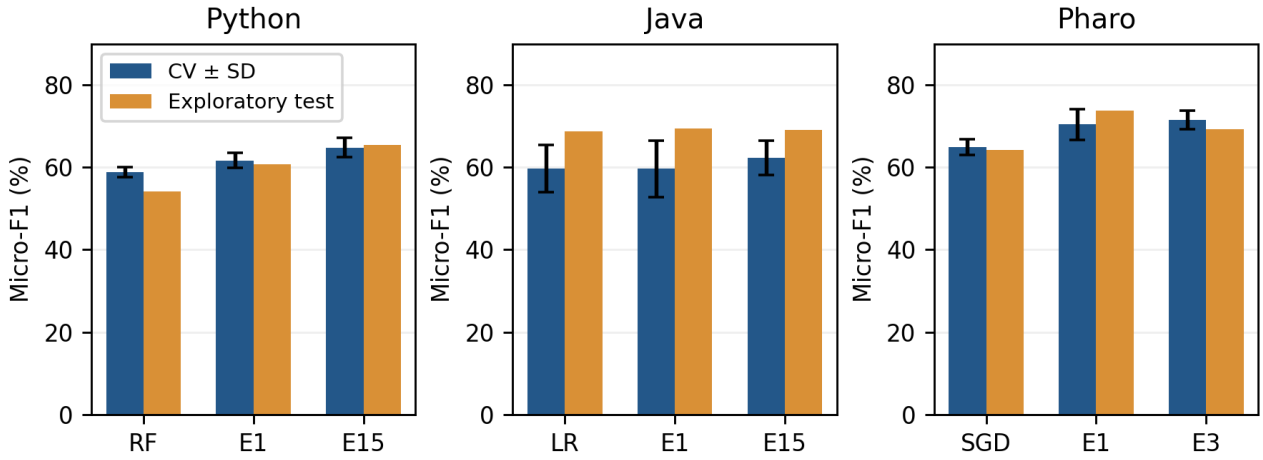


Figure 1: CV-selected ML, E1, and CV-selected DL for each language. CV bars show fold mean and sample SD; final-test bars describe the same fixed methods on a different partition. They do not measure an independent generalization gap.

Table 3: All completed Stage-2 evaluations included in this paper (%). Columns are micro-F1 (μ), macro-F1 (M), and Hamming loss (HL, lower is better). Each score comes from one fixed split and one seed; test results are exploratory.

	Python			Java			Pharo		
Method	μ	M	HL	μ	M	HL	μ	M	HL
LR	56.75	52.44	19.53	68.71	73.44	10.27	64.00	56.53	11.69
Chain	55.77	52.75	19.45	69.64	74.70	10.04	61.61	51.97	12.74
SVM	57.04	52.95	19.06	68.38	73.35	10.15	63.08	56.01	11.69
SGD	56.90	52.68	19.53	69.50	74.41	10.15	64.20	57.88	11.77
RF	54.21	50.19	19.53	63.60	64.95	12.16	64.47	56.07	11.36
E1	60.74	55.47	16.56	69.47	69.46	9.45	73.71	67.20	8.28
E3	59.00	54.96	17.27	68.39	71.11	10.09	69.19	60.15	9.90
E9	62.85	56.34	15.70	67.62	73.71	10.68	71.57	60.58	9.09
E15	65.41	59.65	14.37	69.14	72.84	9.33	70.50	61.94	9.17
E17	62.48	56.69	14.92	70.34	73.42	9.21	70.80	62.83	9.17

test examples yield 24 false negatives and 11 false positives for E15. The observed error counts indicate low recall on this label; they do not establish a specific semantic cause.

The differing supports also limit interpretation. A few changed predictions can substantially affect F1 for a rare category, while micro-F1 is more influenced by frequent positive targets. Macro-F1 complements that aggregate view, but neither averaging scheme guarantees improvement for every category. Consequently, reporting only the highest micro-F1 would conceal relevant documentation types on which the selected model is weaker.

6 Discussion and threats to validity

Method selection. No model family leads uniformly. Linear methods deserve inclusion when evaluating contextual encoders, especially given Java's competitive classical results. R-Drop and focal loss are plausible candidates for further evaluation, but current evidence supports language-specific investigation rather than a universal recommendation. Differences between CV and test rankings demonstrate partition sensitivity, not proof that one method is intrinsically unstable.

Internal validity. Reusing the full population for screening and later test designation creates selection dependence. DL checkpoint selection on the scored fold adds further optimism. A confirmatory design would reserve test groups before every model decision and perform selection only within development data, or use properly nested grouped CV. That design is a proposal, not an experiment completed here. Similarly, no new threshold search or ensemble result is claimed.

Construct validity. Micro-F1, macro-F1, HL, and AP describe different properties. They do not directly measure documentation usefulness, calibration, or downstream developer productivity. The new sentence-level partitions differ from the original per-category competition splits, and our feature set differs from the official RF feature set. Consequently, the study cannot establish that it beats the official competition baselines. Existing ML curve files measure train Hamming loss under repeated fitting budgets, not a logistic or hinge optimization trace; they are not treated as DL-comparable loss curves.

External and statistical validity. Exact-text grouping does not prevent near duplicates or related classes/projects from crossing partitions. The experiments therefore do not test project-disjoint or unseen-language transfer. There is one base seed, no repeated-seed evaluation, and no significance test. Fold SD measures variation across the particular folds rather than uncertainty from all training randomness. In particular, the recorded Linear SVM did not explicitly set its solver-level random state. CPU/GPU and diagnostic overhead also prevent interpreting total duration as a fair efficiency benchmark.

Evidence traceability. All 102 archived metric files,

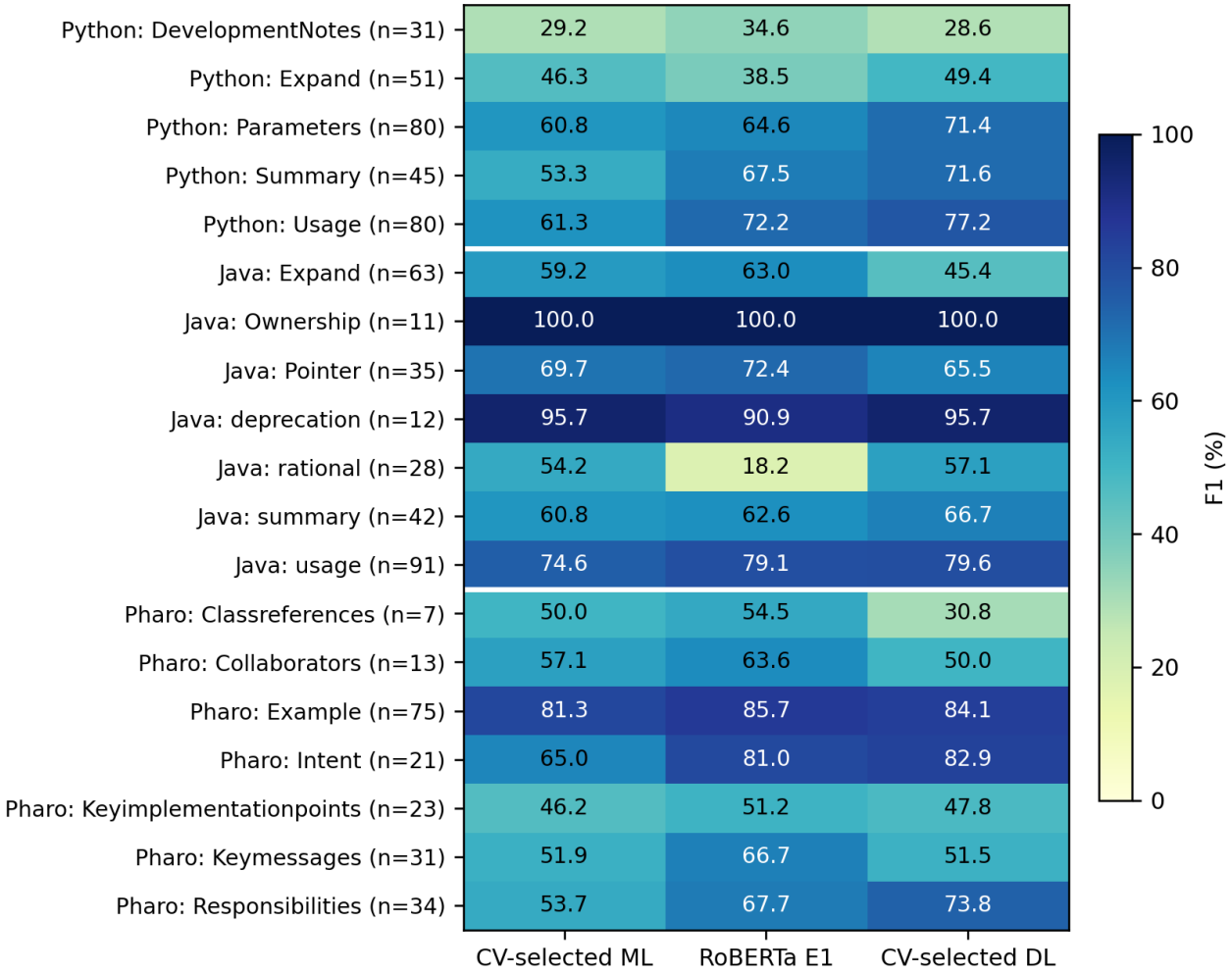


Figure 2: Test F1 per label (%) for the CV-selected ML and DL methods and E1. Selected ML is RF/LR/SGD for Python/Java/Pharo; selected DL is E15/E15/E3. Here n is positive test support, not the training population. The columns are fixed before examining these scores.

including the excluded Dummy controls, were checked against their recorded hashes. The paper's tables use 96 included evaluations: 66 CV and 30 final. Fold means and sample SD were recomputed, shared fold assignments and population fingerprints were verified, and summaries were checked against raw measurements. These checks support transcription consistency, but do not remove the evaluation-design limitations.

7 Conclusion

This study compares five classical methods and seventeen Transformer configurations for multi-label code-comment classification on NLBSE'23. CodeBERT with R-Drop leads screening on Python and Java, while RoBERTa with focal loss leads Pharo. Classical models remain competitive on Java, and per-label results show that aggregate gains can hide regressions. The recorded test outcomes provide exploratory evidence under a shared protocol; independent confirmation requires a test partition reserved before selection, repeated seeds, and evaluation at the desired project or language transfer level.

References

[1] R. Kallis, M. Izadi, L. Pascarella, O. Chaparro, and P. Rani, "The NLBSE'23 Tool Competition," in *Proc. 2nd International Workshop on Natural Language-Based Software Engineering*, 2023. Competition description.

[2] P. Rani, S. Panichella, M. Leuenberger, A. Di Sorbo, and O. Nierstrasz, "How to identify class comment types? A multi-language approach for class comment classification," *Journal of Systems and Software*, vol. 181, art. 111047, 2021. doi:10.1016/j.jss.2021.111047.

[3] NLBSE'23 organizers, "Code comment classification: dataset and baseline classifiers," 2023. Official dataset and baseline repository.

[4] J. Read, B. Pfahringer, G. Holmes, and E. Frank, "Classifier chains for multi-label classification," *Machine Learning*, vol. 85, pp. 333–359, 2011. doi:10.1007/s10994-011-5256-5.

[5] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[6] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, pp. 1536–1547, 2020. doi:10.18653/v1/2020.findings-emnlp.139.

[7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. ICCV*, pp. 2980–2988, 2017.

[8] X. Liang et al., "R-Drop: Regularized dropout for neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 10890–10905, 2021.